THE DATA ENGINEER LAB

DATABRICKS FROM ZERO TO HERO

A 30-Day Interview & Career Readiness Series

DAY 7

Delta Lake Fundamentals — Why Plain Parquet Isn't Enough

SECTION 01

The Problem With Plain Parquet

Traditional data lakes store data as plain files, commonly Parquet, directly in cloud object storage. This is cheap and flexible, but it comes with reliability problems that only show up once you're running real, production pipelines.

- Failed partial writes: if a job fails halfway through, you can end up with a corrupt, half-written dataset with no clean way to know what succeeded.
- Concurrent write corruption: if two jobs write to the same location at the same time, the result can be inconsistent or corrupted data.
- Silent schema changes: nothing stops a new file with a completely different, incompatible schema from being dropped into the same folder.
- Slow full-table scans: without indexing or file-level statistics, engines often have to scan far more data than necessary to answer a query.

CORE CONCEPT

Plain Parquet files have no built-in transactional guarantees. If a Spark job fails halfway through writing, you get a corrupt, half-written dataset, and if two jobs write simultaneously, you get data corruption. This is the reliability gap Delta Lake was built to close.

SECTION 02

What Delta Lake Actually Is

Delta Lake is an open-source storage layer that adds ACID transactions, schema enforcement, time travel, and Change Data Feed to files stored in cloud object storage. It does not replace Parquet; it sits on top of it.

IMPORTANT NUANCE

Delta Lake does not change the underlying Parquet file format. Data is still stored as regular, Snappy-compressed Parquet files that any Parquet-compatible engine can technically read. The real difference is the transactional guarantees, metadata management, and indexing that Delta adds on top of those same files through its transaction log.

The Transaction Log

At the heart of Delta Lake is the transaction log, stored in a hidden folder called _delta_log alongside your data files. Every write, update, or delete is recorded here as an ordered, atomic entry, and this log, not the raw files themselves, is treated as the source of truth for the table's current state.

SECTION 03

How the Transaction Log Delivers Reliability

When a write operation happens, Delta Lake writes the new Parquet data files to storage first. Only after all files are safely written does it add a new entry to the transaction log confirming the change. If the job fails partway through, no new log entry is created, so readers never see a half-finished write.

```
your_delta_table/
  part-00000-....snappy.parquet
  part-00001-....snappy.parquet
  _delta_log/
    00000000000000000000.json   <- initial commit
    00000000000000000001.json   <- next commit (add/remove files)
    00000000000000000002.json   <- next commit
    ...
```

- Atomicity: a write either fully commits to the log, or it doesn't happen at all from the reader's perspective.
- Consistency: the table always reflects a valid, complete state as defined by the log.
- Isolation: concurrent operations don't interfere with or corrupt each other.
- Durability: once a commit is written to the log, it is permanent.

SECTION 04

What This Unlocks

Capability	What It Means in Practice
ACID Transactions	Writes either fully succeed or fully roll back, eliminating partial or corrupted writes.
Schema Enforcement	Delta rejects incoming data that doesn't match the table's defined schema, preventing silent data quality issues.
Time Travel	You can query a table as it existed at a previous version or timestamp, since every historical state is recoverable from the log.
MERGE / Upserts	You can update or delete specific records directly, similar to a database UPDATE statement, instead of rewriting entire files.

SECTION 05

Plain Parquet vs Delta Lake

```
  PLAIN PARQUET LAKE                DELTA LAKE
  +----------------------+          +----------------------+
  |  part-000.parquet    |          |  part-000.parquet    |
  |  part-001.parquet    |          |  part-001.parquet    |
  |  part-002.parquet    |          |  part-002.parquet    |
  |                      |          |  _delta_log/         |
```

```
|  No log. No history.   |          |    00000...json       |
|  No guarantees.        |          |    00001...json       |
|                        |          |  (source of truth)    |
+------------------------+          +-----------------------+
Failed write = corrupt data         Failed write = no new commit
No schema checks                    Schema enforced on write
No history                          Full version history (Time Travel)
```

REMEMBER THIS FOR INTERVIEWS

Delta Lake is not a new file format replacing Parquet. It is a transactional metadata layer, built around a log, that sits on top of Parquet files and makes them behave like a reliable, versioned database table.

SECTION 06

Glossary — Day 7 Terms

Term	Definition
Delta Lake	An open-source storage layer adding ACID transactions, schema enforcement, and time travel on top of Parquet files.
Transaction Log (_delta_log)	The ordered, append-only log of every change made to a Delta table; the source of truth for its current state.
ACID	Atomicity, Consistency, Isolation, Durability; the four guarantees that make database transactions reliable.
Schema Enforcement	Delta's rejection of writes that don't match a table's defined schema, preventing silent data quality issues.
Time Travel	The ability to query a Delta table as it existed at a previous version or timestamp.
Commit	A single, atomic entry in the transaction log representing one completed write operation.

SECTION 07

Interview Questions & Answers

Q1. What problem does Delta Lake solve that plain Parquet doesn't?

A. Plain Parquet files have no built-in transactional guarantees, so failed jobs can leave corrupt, half-written data, and concurrent writes can corrupt a table. Delta Lake solves this using a transaction log that ensures writes are atomic and consistent.

Q2. Does Delta Lake replace Parquet as a file format?

A. No. Data is still stored as regular Parquet files. Delta Lake adds a transaction log and metadata layer on top of those same files, rather than introducing a new file format.

Q3. What is the _delta_log, and why does it matter?

A. It is the hidden folder containing the ordered, append-only transaction log of every change made to a Delta table. It is treated as the source of truth for the table's state, not the raw Parquet files themselves.

Q4. How does Delta Lake prevent partial writes from corrupting a table?

A. New data files are written to storage first, and only after all files are safely written does Delta Lake add a new commit entry to the transaction log. If a job fails partway through, no new commit is created, so readers never see an incomplete write.

Q5. What is Time Travel in Delta Lake?

A. The ability to query a table as it existed at a previous version or timestamp, made possible because the transaction log preserves the full history of changes rather than only the current state.

Q6. What does ACID stand for, and why does it matter for data lakes?

A. Atomicity, Consistency, Isolation, and Durability. Together they guarantee that writes either fully succeed or fully roll back, the table always reflects a valid state, concurrent operations don't interfere with each other, and committed changes are permanent.

STAY ON TRACK FOR ALL 30 DAYS

Comment the keyword **DATABRICKS** on the reel to get this document and future deep-dives sent directly to you.

Piyush Varma | The Data Engineer Lab